CAN-ETH Gateway — Testing Guide

Version 1.0 — Step-by-step instructions for first-time setup, sending CAN frames, and running all tests.

Chapter 1: What Is This Device?

The CAN-ETH Gateway is a small hardware device that connects a **CAN bus** (the communication network used inside vehicles and industrial machines) to a regular **Ethernet network**.

Once it is connected and powered on, you can do the following from any PC on the same network:

- **Send CAN frames** to the device over Ethernet — the device puts them on the CAN bus
- **Receive CAN frames** from the bus — the device sends them to your PC over Ethernet
- **View live J1939 data** (engine speed, temperature, diagnostics) through a web browser or a GUI application
- **Run automated tests** to verify the device is working correctly

Everything is controlled from a PC using Python scripts and a web browser. No special CAN hardware is required on the PC side.

Chapter 2: What You Need

2.1 Hardware

Item	Purpose
The CAN-ETH Gateway device	The device being tested
Ethernet cable (RJ-45)	Connect the device to your PC or network switch
PC with an Ethernet port	Windows or Linux
Two CAN transceivers + short wire (optional)	Only needed for CAN loopback tests
Two 120 Ohm resistors (optional)	Only needed for CAN loopback tests

Important: The first four tests in this guide (Chapters 5–8) require only the Ethernet cable. The CAN wire loopback (Chapters 9–10) is only needed if you want to test the physical CAN bus.

2.2 Software

- **Python 3.10 or newer** (free download)
- **The project files** (the folder you received)
- A **web browser** (Chrome, Firefox, or Edge)

Chapter 3: Install Python

Skip this chapter if Python is already installed.

Windows

1. Open your web browser and go to: `https://www.python.org/downloads/`
2. Click the big yellow **"Download Python 3.x.x"** button
3. Run the downloaded installer
4. **IMPORTANT:** On the first screen, tick the box that says **"Add Python to PATH"** before clicking Install
5. Click **"Install Now"**
6. When it finishes, click **"Close"**

Verify Python is installed

Open a **Command Prompt** (press Windows key, type `cmd`, press Enter) and type:

```
python --version
```

You should see something like `Python 3.12.0`. If you see an error, restart your computer and try again.

Chapter 4: Install the Required Python Packages

The test scripts need a few extra Python packages. Install them once using the following steps.

Open a **Command Prompt** and navigate to the project folder:

```
cd C:\path\to\your\project\folder
```

Then run:

```
pip install requests PyQt5 pyqtgraph
```

Wait for it to finish. You should see `Successfully installed ...` at the end.

That's all. No special CAN drivers, no Wireshark, nothing else needed for the basic tests.

Chapter 5: Connect and Find the Gateway

5.1 Connect the hardware

7. Plug the **Ethernet cable** from the gateway device into your PC's Ethernet port (or a network switch that your PC is also connected to)
8. Power on the gateway device

9. Wait about 5 seconds for it to start up

5.2 Find the gateway's IP address

The gateway has a fixed (static) IP address. The default is:

```
121.145.35.64
```

If this does not work, the device may have been configured to a different IP. Check the label on the device or ask the person who set it up. A common alternative is `10.104.3.64`.

For the rest of this guide, replace `121.145.35.64` with the actual IP of your device.

5.3 Test the connection with a ping

Open a **Command Prompt** and type:

```
ping 121.145.35.64
```

You should see replies like:

```
Reply from 121.145.35.64: bytes=32 time=1ms TTL=64
```

If you see "Request timed out" — see Chapter 12 (Troubleshooting).

5.4 Open the web interface

Open your web browser and go to:

```
http://121.145.35.64
```

You should see the gateway's web dashboard. This page shows:

- Device status and firmware version
- CAN channel settings
- Logging configuration

Leave this browser tab open — you will need it in the next chapter.

Chapter 6: Configure the Gateway

Before running any tests, you need to make sure the gateway is set up to log CAN frames and send them back to your PC. You do this through the web interface you opened in the previous chapter.

6.1 Enable CAN logging on Channel 1 (required for all tests)

In the web interface, find the **Logging** section and set the following:

Setting	Value	Meaning
`logging.ch1.enabled`	1	Turn on logging for CAN channel 1
`logging.ch1.target`	0	Send log data over Ethernet (not USB)

6.2 Enable J1939 mode (required for J1939 tests in Chapter 9–11)

In the **CAN** section, set:

Setting	Value	Meaning
`can.j1939`	1	Enable J1939 protocol interpretation
`can.nbrp`	4	Bit timing for 250 kbps (J1939 standard speed)
`can.ntseg1`	59	Bit timing segment 1
`can.ntseg2`	20	Bit timing segment 2
`can.fd_mode`	0	Use classic CAN, not CAN FD

After changing any setting, click **Save** (or the equivalent button on the page) and **reboot** the device if prompted.

Chapter 7: Test 1 — Send One CAN Frame and See It

This is the first real test. You will send one CAN frame to the gateway and watch it come back to you as a log message.

What happens

```
Your PC  -->  UDP port 4000  -->  Gateway  -->  CAN bus (FDCAN2)
                                                      |
Your PC  <--  UDP port 47808  <-- Gateway  <--  CAN bus (FDCAN1)
```

You send a frame, the gateway puts it on the CAN bus, and — because FDCAN1 and FDCAN2 are physically connected together — FDCAN1 picks it up and sends it back to you as a log message.

Step 1 — Open two Command Prompt windows

You need two Command Prompt windows open at the same time.

Step 2 — Start the listener (Command Prompt 1)

In the first window, go to the `Tests\J1939` folder inside the project:

```
cd C:\path\to\project\Tests\J1939
```

Then start the CLOG receiver:

```
python clog_receiver.py
```

You should see:

```
Listening for CLOG on UDP port 47808 …  (Ctrl-C to stop)
```

Leave this window open. It is now listening for frames from the gateway.

Step 3 — Send a frame (Command Prompt 2)

In the second window, go to the `Tests\testing` folder:

```
cd C:\path\to\project\Tests\testing
```

Send one test frame:

```
python can_tx.py --ip 121.145.35.64 --channel 1 --id 0x18FF0080 --ext --j1939 --data
DEADBEEF01020304
```

What each part means:

- `--ip 121.145.35.64` — your gateway's IP address
- `--channel 1` — send the frame out on FDCAN2
- `--id 0x18FF0080` — a standard J1939 CAN ID
- `--ext` — 29-bit extended frame (required for J1939)
- `--j1939` — mark it as a J1939 frame
- `--data DEADBEEF01020304` — 8 bytes of test data

You should see in Command Prompt 2:

```
Gateway : 121.145.35.64
Mode    : UDP
Frame   : CH=FDCAN2  ID=0x18FF0080 [EXT|J1939]  len=8  data=DEADBEEF01020304
Count   : 1  interval: 10.0 ms

  [    1/1]  sent

Done — 1 sent, 0 failed.
```

Step 4 — Check the listener

Go back to **Command Prompt 1**. You should see a new line appear:

```
  [  3.141592] J1939  seq=1   SA=0x80  DA=0xFF  PGN=0x0FF00  DLC=8
data=deadbeef01020304
```

This means it worked. The frame left your PC, travelled through the gateway, went onto the CAN bus, came back through the other CAN channel, and arrived back at your PC as a logged frame.

If nothing appears — see Chapter 12 (Troubleshooting).

Chapter 8: Test 2 — Send Multiple Frames

Once Test 1 works, you can send multiple frames to check for reliability.

Send 100 frames in a row:

```
python can_tx.py --ip 121.145.35.64 --channel 1 --id 0x18FF0080 --ext --j1939 --data
AABBCCDD11223344 --count 100 --interval 0.05
```

You should see 100 frames appear in the listener window. When you press **Ctrl-C** to stop the listener, it prints a summary:

```
Received 100 frames in 5.1s  (19.6 fps)
```

If the received count is lower than 100, frames were lost. See Chapter 12.

Chapter 9: Test 3 — CAN Loopback Reliability Test

This test sends many frames and checks that every single one comes back correctly. It measures frame loss, timing, and reliability.

Physical setup required

You need to connect FDCAN1 and FDCAN2 together with a short wire:

```
FDCAN2 connector  ─── CAN-H wire ───  FDCAN1 connector
FDCAN2 connector  ─── CAN-L wire ───  FDCAN1 connector

Add one 120Ω resistor across CAN-H and CAN-L at EACH connector end.
```

Check the device label or datasheet for the exact pin numbers.

Run the test

Open a **Command Prompt**, go to the `Tests\testing` folder, and run:

```
python can_loopback.py --ip 121.145.35.64 --count 1000
```

The test sends 1000 frames and checks that every one comes back. You will see a progress report and a final result:

```
Sent 1000, received 1000, lost 0  (0.0%)  avg latency 2.1 ms
PASS
```

If any frames are lost, the test prints `FAIL` and shows which sequence numbers are missing.

Chapter 10: Test 4 — J1939 Automatic Test Suite (5 Tests)

This is a set of five automated tests that check all parts of the J1939 protocol. They run in sequence and each prints a PASS or FAIL result.

What is tested

Test	What it checks
Test 01	Gateway announces itself on the bus (Address Claim)
Test 02	Gateway responds to a PGN data request
Test 03	Gateway broadcasts live PGN data
Test 04	Gateway handles large messages (Transport Protocol / BAM)
Test 05	Gateway reports diagnostic trouble codes (DM1)

Physical setup required

Same as Chapter 9: FDCAN1 and FDCAN2 connected together with 120 Ohm termination.

Also check these gateway settings in the web interface:

Setting	Value
`logging.ch1.enabled`	1
`logging.ch2.enabled`	1
`logging.ch1.target`	0 (Ethernet)
`logging.ch2.target`	0 (Ethernet)
`can.j1939`	1

Edit the config file

Open the file `Tests\J1939\udp_loopback\config.py` in any text editor (Notepad is fine) and check that the gateway IP is correct:

```
GW_IP = '121.145.35.64'   # <-- change this if your gateway IP is different
```

Save the file.

Run the test suite

Open a **Command Prompt**, go to the `Tests\J1939\udp_loopback` folder, and run:

```
python run_all.py
```

The tests run one by one. Test 01 will ask you to reboot the gateway — follow the instructions on screen.

When finished, you will see a summary table:

```
╔══════════════════════════════════════════════════════╗
  Results
╠══════════════════════════════════════════════════════╣
  01  Address Claim (loopback)              PASS ✓
  02  PGN Request → reply (loopback)        PASS ✓
  03  Broadcast PGN EEC1 (loopback)         PASS ✓
  04  TP / BAM reassembly (loopback)        PASS ✓
  05  DM1 Active Diagnostics (loopback)     PASS ✓
╚══════════════════════════════════════════════════════╝
```

To run only specific tests, add the test numbers after the command. For example, to run only tests 1 and 3:

```
python run_all.py 1 3
```

Chapter 11: Test 5 — Validation GUI (No CAN Wiring Needed)

The validation GUI is a graphical application that connects to the gateway over Ethernet and shows live data on multiple pages. It also runs J1939 simulations to populate the display with realistic data — **no CAN wiring is needed** for this test.

What you will see

The application has several tabs:

- **Gauges** — live engine speed, temperature, fuel economy (simulated)
- **Diagnostics** — active and historical fault codes
- **Network Nodes** — J1939 ECUs discovered on the bus

Run the GUI

Open a **Command Prompt**, go to the `validation\demo` folder, and run:

```
python main.py
```

A window will open. It automatically connects to the gateway at `121.145.35.64`. If your gateway IP is different, you can pass it as an argument:

```
python main.py --ip 10.104.3.64
```

Watch the Gauges page — the values should update every 100 ms with simulated J1939 data being sent through the gateway.

Chapter 12: Test 6 — Full J1939 Validation Test Suite

This is the most complete automated test. It runs three simulators at the same time:

- **J1939 Telemetry** — two simulated engine ECUs sending live data (Gauges page)
- **DM1/DM2 Diagnostics** — simulated fault codes cycling through scenarios (Diagnostics page)
- **Address Claim** — simulated ECU node discovery (Network Nodes page)

No CAN wiring needed

This test uses **channel 255** (software simulation), which means the frames are injected directly into the gateway's internal data store without going through the physical CAN bus. You only need the Ethernet cable.

Edit the config file

Open `validation\j1939\config.py` in a text editor and check the gateway IP:

```
GW_IP = '121.145.35.64'   # <-- change this if needed
```

Run the test

Open a **Command Prompt**, go to `validation\j1939`, and run:

```
python run_all_tests.py
```

To use software simulation (no CAN hardware):

```
python run_all_tests.py --channel 255
```

To specify the gateway IP:

```
python run_all_tests.py --ip 10.104.3.64 --channel 255
```

The simulators run for approximately 30 seconds, then print a result. While they run, open the validation GUI in Chapter 11 to see the data updating live.

Chapter 13: Summary of All Tests

#	Test	Folder	Command	CAN Wiring?
1	Send one frame	`Tests\testing`	`python can_tx.py --`	Yes

			`ip ... --channel 1 --id 0x18FF0080 --ext --j1939 --data DEADBEEF01020304`	
2	Listener	`Tests\J1939`	`python clog_receiver.py`	No
3	Loopback reliability	`Tests\testing`	`python can_loopback.py --ip ...`	Yes
4	J1939 suite (5 tests)	`Tests\J1939\udp_loopback`	`python run_all.py`	Yes
5	Validation GUI	`validation\demo`	`python main.py`	No
6	J1939 validation suite	`validation\j1939`	`python run_all_tests.py --channel 255`	No

Chapter 14: Troubleshooting

"ping 121.145.35.64 — Request timed out"

The PC cannot reach the gateway. Possible causes:

10. **Wrong IP** — check the device label or web interface
11. **Different subnet** — your PC may be on a different network. If your PC's IP starts with `192.168.1.x`, try `192.168.1.100` (bootloader address) or check the web interface
12. **Cable not connected** — make sure the Ethernet cable is plugged into both the gateway and the PC
13. **Device not powered** — check that the device's power LED is on
14. **Firewall** — temporarily disable the Windows Firewall and try again

"python can_tx.py — sent, but nothing appears in clog_receiver.py"

15. Check that `logging.ch1.enabled = 1` in the web interface
16. Check that `logging.ch1.target = 0` (Ethernet, not USB)
17. Check that FDCAN1 and FDCAN2 are wired together (the frame needs to travel physically from FDCAN2 to FDCAN1)
18. Make sure you are running `clog_receiver.py` BEFORE sending the frame

"J1939 test suite fails at Test 01 (Address Claim)"

19. Check `can.j1939 = 1` in the web interface
20. Test 01 asks you to reboot the gateway — make sure you do this when prompted
21. Check that `can.nbrp = 4` for 250 kbps (J1939 standard)

22. Check the CAN termination resistors (120 Ohm at each end of the bus)

"pip install fails — no internet connection"

If the PC does not have internet access, you need to transfer the packages manually:

23. On a PC with internet access, run: `pip download requests PyQt5 pyqtgraph -d packages`
24. Copy the `packages` folder to the test PC
25. On the test PC, run: `pip install --no-index --find-links=packages requests PyQt5 pyqtgraph`

"The validation GUI opens but shows no data"

26. Check that the gateway IP is correct (edit `validation\j1939\config.py`)
27. Check that the gateway web interface is reachable at `http://121.145.35.64`
28. Run the J1939 simulation in a separate window: `python run_all_tests.py --channel 255`
29. Switch to the **Gauges** tab in the GUI — data should appear within 5 seconds

"I see CLOG status frames but no CAN frames"

Status frames (`STATUS seq=... fw=...`) are heartbeat messages sent every second by the gateway. They confirm the gateway is alive and connected. CAN frames only appear when you send something. Run `can_tx.py` in a separate window.

Quick Reference Card

Gateway IP (default): `121.145.35.64`

Send one frame:

```
python Tests\testing\can_tx.py --ip 121.145.35.64 --channel 1 --id 0x18FF0080 --ext --
j1939 --data DEADBEEF01020304
```

Listen for frames:

```
python Tests\J1939\clog_receiver.py
```

Run J1939 test suite:

```
python Tests\J1939\udp_loopback\run_all.py
```

Run validation GUI:

```
python validation\demo\main.py
```

Run full validation suite (no CAN wiring):

```
python validation\j1939\run_all_tests.py --channel 255
```